

Del 1-Rapport: Continuous Integration och testautomatisering

Vad är Continuous Integration (CI)?

Continuous Integration (CI) är ett arbetssätt inom mjukvaruutveckling där kodändringar kontinuerligt integreras i ett gemensamt kod-repo. Ett kod-repo (förkortning av 'code repository') är helt enkelt en plats där källkod lagras och hanteras.

Varje gång en utvecklare eller testutvecklare gör en kodändring, pushar, gör en 'push request' eller gör en aktion som är definierat som trigger i YAML-filen, triggas en CI-pipeline. Den körs sedan automatiska processer som bygger applikationen och kör tester. Resultatet syns direkt i CI-verktyget.

Syftet med CI är att upptäcka fel så tidigt som möjligt. Genom att testa ofta minskar risken för att problem byggs upp och upptäcks sent i projektet, vilket annars kan bli både tidskrävande och kostsamt.

Ett typiskt exempel är; låt oss säga att på inlämning uppgift Del-2(testautomatisering av Inloggningsfunktion & Del-3 (Automatisering av integrationstester), jobbar en testutvecklare med Del-2 och den andra jobbar med Del-3. Utan CI jobbar testutvecklarna var för sig i flera dagar och till slut slår ihop koden. Detta kan orsaka konflikter, buggar och kaos, plus att testerna måste manuellt testa att allt fortfarande fungerar.

Å andra hand, med CI pushar testutvecklarna små ändringar flera gånger per dag. Varje push eller pull request trigger automatiskt att projektet byggs, testerna körs och resultat visas-både om något har gått sönder och direkt om ändringar fungerar tillsammans.

Resultatet blev att koden hålls stabil, buggar upptäcks tidigt och teamet slipper stora "merge crashes".

Fördelar och utmaningar med CI

Fördelarna med CI är bland annat återkoppling på kodändringar, stabilare kodbas och minskad risk för integrationsproblem. Utvecklare får snabbt veta om deras kod orsakar fel.

Utmaningar med CI är att det kräver initial konfiguration och underhåll. Om tester är instabila kan de ge falska fel, vilket kan minska förtroendet för CI-systemet. Långa testsviter kan även leda till längre byggtider.

Vad är testautomatisering?

Testautomatisering innebär att tester körs automatiskt av en maskin istället för manuellt av en person. Testerna skrivs som kod och kan köras om och om igen utan extra arbetsinsats. Testautomatisering används för att automatiskt testa funktionalitet, API:er, UI, Integrationer och Prestanda.

Testautomatisering är en central del av CI eftersom CI bygger på att tester körs automatiskt vid varje kodändring. Utan automatiserade tester kan CI inte ge snabb och tillförlitlig feedback.

I Python-projekt används ofta pytest som testverktyg eftersom det är snabbt, lätt att använda och väl anpassat för CI-miljöer. För JAVA-projekt används JUnit som testverktyg och i DotNet-projekt används NUnit eller XUnit som testverktyg.

Det finns många olika typer av automatiserade tester, somliga är;

- **Unit test:** Testar enskilda isolerade class, metoder och funktion. Den testar avgränsad del av koden för exakt det den ska. Syftet med Unit test är upptäcka logiska fel tidigt och ge snabb feedback. Vanliga verktyg som används för Unit test är; NUnit, JUnit, XUnit, pytest etcetera.
- **Integration Test:** Verifierar att flera delar av systemet fungerar korrekt tillsammans. Syftet med integrationstest är att säkerställa att modulens eller systemet interna delar fungerar tillsammans. Ett vanligt verktyg som används för integrationstest är Testcontainers.
- **E2E Test /UI Test:** Verifierar att hela systemet fungerar från början till slut, precis som en riktig användare upplever det. Syftet med E2E test är att testa verkligt beteende genom UI eller API. Vanliga verktyg som används för E2E test är Selenium, Playwright, Cypress etcetera
- **Regression Test:** Det är test som säkerställer att något som fungerar tidigare fortfarande fungerar efter en ändring. Syftet med regression test är att verifiera att ny kod inte förstört gamla funktioner. En kombination av olika testverktyg används för att exekvera Regression test.

Hur körs tester automatiskt i GitHub Actions?

För att tester ska köras automatiskt i GitHub Actions krävs följande grundläggande steg:

1. Ett projekt/repository som innehåller kod och tester
2. Ett workflow skapas i mappen `.github/workflows/ci.yml`

3. Definiera följande i ci.yml fil
 - Triggers ->
 - Jobs ->
 - Stage->
 - i. Build/Setup->
 - ii. Test ->
4. Koden pushas till GitHub Actions. När en utvecklare gör en push eller skapar en pull request, triggas GitHub Actions.
5. Ett workflow startas. GitHub läser workflow-filen som ligger i mappen .github/workflows.
6. Källkoden checkas ut. Repo laddas ner till CI-miljön med actions/checkout
7. Rätt miljö sätts up. Exempelvis installeras Python med actions/setup-python eller .NET med actions/setup-dotnet eller JAVA med actions/setup-java.
8. Beroende installeras. Projektets beroende installeras, till exempel via pip install -r requirements.txt
9. Tester körs automatiskt. Tester som skapade körs automatiskt med ett kommando, till exempel pytest. GitHub skapar och kör inte några tester åt utvecklarna. Den endast körs det kommando i det skapade projekt som kör testerna. Till exempel pytest.
10. Resultatet utvärderas
 - Om alla tester passerar -> workflowet godkänns
 - Om ett test fallerar -> workflowet misslykas.
11. Kort exmple (yaml):
 - uses: actions/checkout@v4
 - uses: actions/setup-python@v4
 - with:
 - python-version: "3.10"
 - run: pip install -r requirements.txt
 - run: pytest
 -

Skillnaden mellan CI, Continuous Delivery och Continuous Deployment

Jämförelsetabell CI vs Continuous Delivery vs Continuous Deployment			
Aspekt	CI	Continuous Delivery	Continuous Deployment
Mål	Upptäcka fel tidigt	Alltid releasabel kod	Automatisk produktionrelease
Manuellt steg	Nej (Push eller Pull request är manuellt)	Ja (release-godkännande)	Nej (full automation)
Produktionrelease	Aldrig	Vid manuellt godkännande	Automatisk vid grönt bygge
Testkrav	Enhets- & integrationstester	Acceptanstester i staging	Full regression
Riskenivå	Låg	Medel	Hög (Kräver mogenhet)
Feedbackcykel	Minuter → Timmar	Timmar → Dagar	Sekunder → Minuter
Typiska verktyg	Jenkins, GitLab CI	ArgoCD, Spinnaker	Kubernetes, Flux, GitHub Actions

Continuous Integration (CI) är grunden i ett automatiserat utvecklingsflöde. Det handlar om att bygga och testa koden helt automatiskt varje gång någon gör en ändring. På så sätt upptäcks fel tidigt och man säkerställer att ny kod fungerar tillsammans med det som redan finns. Målet med CI är helt enkelt att hålla kodbasen stabil och undvika överraskningar längre fram.

Continuous Delivery bygger vidare på detta. Här är systemet alltid ett läge där det går att släppa en ny version. Alla tester är körda och godkända och applikationen är färdigpaketerad för produktion. Skillnaden mot CI är att själva deployment krävs ett manuellt knapptryck, till exempel efter ett godkännande från ansvariga person eller verksamheten. Feedbackcykeln är också längre än feedbackcykeln för CI samt olika verktyg används för både CI och Continuous Delivery.

Continuous Deployment tar automatiseringen ännu ett steg längre. När allting har testat och godkänts (CI -CD = OK and approved) sker driftsättningen automatiskt utan att någon behöver trycka på en knapp. Detta betyder att varje ändring som klarar testerna går hela vägen till produktion. För att detta skall kunna fungera krävs välbyggda tester, hög testtäckning och stabila processer men det positiva med detta är att nya funktioner nå användarna mycket snabbare. Inom bilbranschen som jag arbetar i är detta bara en dröm som är nästa omöjligt att införa fullt ut i praktiken.

Kort sagt handlar skillnaden mellan CI, Continuous Delivery och Continuous Deployment om hur långt automatiseringen sträcker sig. CI fokuserar på kvalitet och stabilitet, Continuous Delivery ser till att systemet är redo för release när som helst och Continuous Deployment automatiserar hela vägen till produktion.

Analys av Testverktyg i CI-miljöer kopplat till projektets kod (Del-2 & Del-3)

I projektet används flera testverktyg som kompletterar varandra i CI-miljön. Tanken är att få så bra testtäckning som möjligt utan CI-flödet blir långsamt eller instabilt. Valet av testverktyg påverkar därför både vilka fel som kan upptäckas och hur snabbt man får återkoppling när kod ändras samt hur mycket tid man lägga för att felsöka. Här kommer jag att analysera Selenium + Pytest vs Playwright + Pytest och Pytest vs Unittest.

Både Playwright + Pytest och Selenium +Pytest används för automatiserad testning av webbapplikationer i Python. Skillnade ligger inte i vad dom tester utan i hur mycket arbete som krävs för att få stabila och pålitliga tester.

Pytest och Unittest används båda för att skriva och köra tester i Python, men de är byggda med olika mindset. Unittest prioriterar struktur och formalism medan Pytest prioriterar läsbarhet och flexibilitet.

Playwright + Pytest

Playwright är ett modernt verktyg för webbläsarautomatisering. Tillsammans med Pytest bildar det en lättviktig och mycket stabil teststack.

Fördelar med Playwright + Pytest

- Tester beter sig stabilt utan att man hela tiden behöver fundera på väntetider
- Felsökning tar mindre tid eftersom tester inte faller slumpmässigt
- CI-körning blir jämnare och mer förutsägbara
- Man kan lägga energin på vad som testas och inte på får testerna att överleva eller felsöka flaky tester.
- Snabba körningar både lokalt och i CI (jag gjorde en jämförelse och tid skillnaden för körning av 15 testfall med både selenium och playwright was cirka 35 sekunder)

Nackdelar med Playwright + Pytest

- Ingen riktig Safari, bara Webkit. Playwright kan köra Webkit, vilken ligger nära safari men det är inte samma sak som att testa i Apples faktiska Safari-browser. Detta har själv inte testat men dom flesta som har testat detta tycker samma.
- Nyare verktyg med kortare historik än Selenium.
- Mindre community och färre färdiga svar.

Selenium + Pytest

Selenium är ett välkänt och välprövat verktyg. Tillsammans med Pytest blir det betydligt mer användarvänligt än Selenium med Unittest.

Fördelar med Selenium + Pytest

- Mycket moget community: Selenium har använts i många år och är välbeprövat i produktion.
- Stöd för all större webbläsare: Inklusive riktig Safari, vilket fortfarande är ett krav i vissa projekt.
- Stor mängd dokumentation och exempel: Det är lätt att hitta lösningar på vanliga problem.
- Flexibelt ramverk: Går att anpassa till många typer av teststrategier och miljöer.
- Passar bra för långlivade testsviter: I organisationer där Selenium redan är etablerat är det att fortsätta med Selenium.

Nackdelar med Selenium + Pytest

- Kräver mer konfiguration: Drivrutiner, versioner och miljöer måste hanteras aktivt.
- Manuell vänte logik: Testaren måste själv hantera timing och synkronisering.
- Mer tid går till stabilisering: UI-förändringar och timing problem orsakar ofta instabila tester.
- Tester blir lätt instabila i CI-miljöer: Tester blir längre och mer tekniskt detaljerade.

Pytest-grunden i testautomatiseringen

Pytest används som det grundläggande testramverk i projektet. Det hanterar testkörning, assertioner och struktur, vilket gör det enkelt att skriva tydliga och fokuserade tester.

Exempel från projektet:

```
def test_get_products_returns_200():  
    response = requests.get(f"{BASE_URL}/products")  
  
    assert response.status_code == 200
```

Fördelar med Pytest

Testfallen är korta och lätta att förstå, vilket gör dem enkla att underhålla över tid. De körs snabbt i CI och ger direkt feedback vid varje push. När ett test fallerar är det också lätt att se exakt vad som gått fel. Dessutom är testsviten enkel att bygga vidare på när nya funktioner tillkommer.

Nackdelar med Pytest

Pytest är begränsat till Python-projekt, och om testsviten växer utan tydlig struktur kan den bli svår att överblicka men i ett litet projekt som mitt Del-2&3 projekt är lätt att hantera.

Unittest

Unittest är inspirerat av JUnit och bygger på klassbaserade tester med tydlig livscykel (setUp, tearDown). Strukturen är strikt och förutsägbar

Fördelar med Unittest

- Ingår i Python standardbibliotek
- Passar bra i mycket kontrollerade miljöer
- Lätt att förstå för Java/Junit-erfarna team

Nackdelar med Unittest

- Begränsad parametrisering
- Mindre informativa felrapporter
- Mindre stöd för moderna testflöden

Hur teststrategi påverkar byggtid?

I moderna utvecklingsprojekt är byggtid en viktig del av vardagen. Det är tiden från att kod skickas in tills man vet om något gick sönder. När byggtiden är kort får utvecklare snabb feedback och kan fortsätta arbeta utan avbrott. När den är lång skapar det frustration och bromsar hela teamet samt leveranser till slutkunden.

En av de största faktorerna bakom byggtid är teststrategin.

Alla tester är inte lika dyra. Enhetstester går snabbt och ger tidig feedback. UI-tester och mer omfattande integrationstester tar betydligt längre tid. Om alla tester körs varje gång någon gör en liten ändring, blir byggtiden snabbt väldigt lång.

Detta leder ofta till att utvecklare väntar längre på svar eller börjar ignorera CI-resultat, vilket i sin tur ökar risken för fel längre fram.

En fungerande teststrategi handlar om balans. Snabba tester bör köras tidigt för att fånga enkla fel direkt. Tyngre tester bör köras mer sällan eller vid särskilda tillfällen, till exempel innan en större sammanslagning eller nattetid.

Genom att köra tester i rätt ordning och parallellt där det är möjligt kan byggtiden hållas nere utan att kvaliteten försämras.

Byggtid är sällan ett tekniskt problem i sig. Det är oftast ett resultat av hur testerna används. En genomtänkt teststrategi ger snabbare feedback, mindre frustration och ett arbetsflöde som faktiskt hjälper utvecklarna i stället för att stå i vägen.

Del 2 & 3: GitHub länk: [Ebesoh/InlämningsUppgift](#) TA: [Inlämningsuppgift till Test Automatisering EC Utbildning](#)